Sujet du projet : Conception et déploiement d'une application Web orientée données et API

1. Contexte général

Dans le monde professionnel, le développement d'une application web moderne ne se limite pas à écrire du code. Il implique une **démarche complète**, allant de la conception à la mise en production, en passant par :

- le choix d'une **architecture cohérente** ;
- le développement **front-end et back-end** ;
- la gestion et l'exploitation de **données réelles** ;
- l'utilisation de **bases de données** ;
- la **conteneurisation** et le **déploiement** ;
- la **documentation** et la **justification des choix techniques**.

Ce projet vise à vous placer dans une situation **réaliste**, proche des pratiques industrielles.

2. Organisation du travail

- **Travail en binôme obligatoire**
  (le travail en binôme est obgligatoire pour ce projet, il a pour objectif de refléter un contexte réel de développement)
- Technologies **libres**, tant qu'elles sont cohérentes et justifiées
- Le projet est construit **progressivement** au fil des TME
- Le **rapport est rédigé au fur et à mesure**, et non à la fin uniquement

3. Objectif global du projet

Le but du projet est de **concevoir, justifier, implémenter et déployer** une application web permettant :

- d'afficher un **catalogue de films** ;
- de proposer une **barre de recherche** ;
- de reposer sur une **architecture claire front-end / back-end / base de données** ;
- d'exposer et consommer une **API HTTP** ;
- d'être **conteneurisée et déployable**.

4. Source des données (au choix)

Vous n'êtes **pas obligés** d'utiliser une API open-source.

Option A -- API externe

- Exemple : TMDB

- Lecture de documentation
- Gestion de clé API
- Normalisation des données

Option B -- Dataset existant

- Exemple : Kaggle
  https://www.kaggle.com/datasets
- Import dans une base :
  - SQL,
  - NoSQL,
  - ou combinaison des deux

Vous devez justifier le choix de la source de données.

5. Architecture technique attendue

5.1 Front-end (au choix)

Exemples possibles :

- Astro
- React + Vite
- Angular
- Vue + Vite
- Svelte / SvelteKit

5.2 Back-end (au choix)

Exemples possibles :

- FastAPI (Python)
- Node.js (Express ou NestJS)
- PHP (Laravel ou Symfony)
- ASP.NET Core (C#)
- Go (Fiber ou Gin)

5.3 Base de données (obligatoire)

Vous devez utiliser **au moins une base de données** :

- SQL : PostgreSQL, MySQL, MariaDB
- NoSQL : MongoDB, Redis, Neo4j, Elasticsearch

Les combinaisons sont autorisées.

6. Fonctionnalités minimales attendues

- Catalogue de films dynamique
- Barre de recherche
- Communication front-end / back-end fonctionnelle
- Gestion minimale des erreurs

7. Fonctionnalités avancées (optionnelles mais valorisées)

- Films favoris
- Recommandations de films similaires
- Cache (Redis)
- Recherche avancée (Elasticsearch)
- Pagination

8. Conteneurisation et déploiement (obligatoire)

8.1 Docker

- Conteneuriser :
  - front-end
  - back-end
  - base de données

8.2 Kubernetes

- Déployer sur un cluster
- Utiliser :
  - Pods
  - Services
  - Ingress
  - ConfigMaps
  - Secrets
- Réfléchir au scaling

9. CI/CD et dépôt Git

- Dépôt Git (GitLab ou GitHub)
- Pipeline CI/CD automatisé :
  - build
  - tests (si présents)
  - build des images Docker
  - déploiement

10. Modalités de rendu

Le projet doit être rendu sous la forme d'une **archive** `.zip` contenant :

- l'ensemble du code source ;
- un fichier expliquant **comment lancer le projet** (`README.md` obligatoire) ;
- idéalement :
  - un script unique à exécuter (`run.sh` , `make up` , `docker-compose up` , etc.)
  - permettant de lancer automatiquement l'application.
- un document PDF rapport du projet, reprenant les éléments conçus au fil des TME ;
- une vidéo de présentation (5 minutes maximum, voir ci-dessous).

L'objectif est que l'application puisse être **lancée facilement** sans configuration complexe.

11. Vidéo de présentation avec démonstration

Vidéo de présentation avec démonstration (5 minutes maximum)

- Présentation :
  - de l'architecture,
  - des fonctionnalités principales,
  - du déploiement.
- Démonstration de l'application.

12. Date limite

À rendre avant le 17 Mai 2026 à 23h59

Tout rendu hors délai pourra être pénalisé.

13. Critères d'évaluation

L'évaluation portera notamment sur :

- la cohérence de l'architecture ;
- la qualité des choix techniques ;
- la maîtrise des outils (API, base, Docker, Kubernetes) ;
- la capacité à justifier ;
- la clarté de la documentation / rapport ;
- la qualité de la présentation / démonstration.

14. Esprit du projet

L'objectif n'est pas de produire l'application la plus complexe possible, mais de montrer que vous êtes capables de :

- travailler en équipe ;

- réfléchir avant de coder ;
- faire des choix réalistes ;
- documenter et expliquer votre travail ;
- vous rapprocher des pratiques professionnelles.

Exemple d'architecture possible (non imposée)

À titre **illustratif**, une architecture possible pour ce projet est présentée ci-dessous.

Cet exemple montre une application découpée en plusieurs **conteneurs**, avec :

- un **front-end** jouant le rôle de client ;
- un **back-end API** structuré (API Gateway + Data Access Layer) ;
- une ou plusieurs **bases de données** (SQL, NoSQL, recherche, cache) ;
- un **moteur d'algorithmes** séparé (recommandation, similarité, recherche avancée).

Cette architecture est donnée comme exemple :

- vous n'êtes **pas obligés** de la reproduire à l'identique ;
- vous pouvez la simplifier ou l'adapter ;
- votre propre architecture doit être **cohérente, justifiée et expliquée** dans le rapport.

Schéma d'architecture (exemple) :

